

REPORT

This is the condensed write-up. The full version — complete package inventory, schema field table, per-condition trigger table and the reasoning behind every trade-off below — is at `REPORT.md` in the repository root.

1. Architecture

Eight packages, four apps. `schemas` holds Zod contracts used as runtime validators at four boundaries: model output, artifact files, typed extraction, API surface. `surface` wraps Playwright behind the interface that is the seam for other surfaces (§4). `discovery` runs the model's observe → act loop under a step budget, a time budget and a policy check. `recorder` distills a run log into an artifact. `replay` executes artifacts and cannot reach a model — it has no dependency on `model-provider`. `redaction` masks at the write boundary; `session` owns the handoff state machine. The apps are a deliberately hostile Express target app, the CLI, a NestJS server and a Next.js console; the last two are optional and sit off the path of every core operation.

Replay was built before discovery. The executor was written and tested against hand-written artifacts before the model loop existed, making it testable at zero model cost and leaving discovery only to satisfy an already-proven contract. That ordering is also why the replay half is the solid half (§3). There is no database: artifacts are JSON, run logs JSONL, screenshots PNGs under `evidence/`, and the schemas would move to Postgres `jsonb` unchanged.

2. Artifact schema

`Capability` declares typed `inputs` (secrets by reference — the step carries `{{operatorPassword}}`, never the value), typed `outputs` with coercion targets, ordered `steps` each carrying a locator ladder, a wait condition and a `risk`, plus `checkpoints`, `extractions`, `businessOutcomes`, `expectedDialogs` and `interstitials`. Elements are found by a ladder of one to three ranked strategies, most durable first: accessible name → label-anchored walk → CSS. Which rung resolved is recorded, which is a free drift signal.

Signals on business outcomes are the field the schema most depends on. A rule keyed only on "checkpoint X failed" fires for every reason that checkpoint can fail, since an empty result, a rejected input, an expired session and a 500 all arrive through the same failed assertion. So each rule also declares a `signal` — text the page shows when *this* outcome is the right one. **Risk is recorded, not inferred**, decided at record time from the policy's `irreversibleControlLabels`, because by the time replay sees the button it is already on the page it would change. **approved is earned**: the recorder emits `draft`, and discovery replays the artifact it just wrote, promoting it only if checkpoints hold and outputs match what discovery read off the page. Re-discovery bumps `version` and archives rather than overwrites. **Credentials are placeholders first** — the prompt names inputs but never values, the model types `{{operatorPassword}}` literally, and substitution happens at the page boundary, so the secret is absent from prompt, conversation and log by construction rather than by redaction.

3. Determinism & error handling

Replay walks steps in order, substitutes placeholders, resolves ladders and acts with each step's declared wait. No model is invoked. Outcomes are `success`, `business_outcome` (the flow worked and the answer is negative), `recovered` (outputs reached but not first time, exiting 0 like a success) and `failed`.

The order they are decided in is the contract. A failed checkpoint says the result is missing, never why; deciding business outcomes first — as this classifier originally did — means the first listed rule wins for every cause, and an expired session reaches the caller as "no such member". The order is now: completed → `success` or `recovered`; then failures about the run rather than the answer (`session_expired`, `app_error`, `unexpected_dialog`, `approval_required`, `policy_denied`), which can never be business outcomes; then a

declared rule, firing only if its condition *and* its signal both hold; otherwise `failed`. Nine target-app triggers exercise every condition the brief names, from not-found and validation errors through permission denial, slow grids, dismissible interstitials, undeclared dialogs, 500s and silent session expiry.

What a real discovery produced. `evidence/` holds an unedited artifact Gemini discovered against this flow. Across those nine scenarios it got four right — `success`, `unexpected_dialog`, `app_error`, `session_expired` — and those four are exactly the conditions the *engine* detects by mechanism. Every condition that depended on the artifact declaring something failed, for three distinct reasons. First, `member_not_found` was keyed on a checkpoint that only evaluates if the click succeeded, and not-found is precisely the case where it cannot, so the rule is unreachable by construction; nothing at all was proposed for the validation error or the permission denial, neither having been seen. Second, the click's two strongest rungs are the literal name of the member discovery happened to use, so for another member the ladder drops to its positional rung — correct here, by luck — and for an empty result nothing resolves. Third, the checkpoint guarding the extraction uses the extraction's own locator, so it passes on an empty cell and coercion of `""` then fails.

The dividing line is the finding. Mechanism-detected conditions are artifact-independent and held up against cases they had never seen; declared conditions are only as good as what the model observed, and it observed the happy path. A green table from hand-written rules proves the executor, not the pipeline that has to produce them. §7 proposes a fix for each fault.

Two recoveries exist, cheapest first: a declared interstitial is dismissed and the checkpoint re-read, then a wait-and-retry. Retries are confined to reads, since a click cannot be retried safely when replay cannot know whether the first took effect. Detection that does not depend on the artifact covers auth pages (URL shape, or a password field on screen), app errors (response status, since a 500 renders fine and satisfies text assertions) and native dialogs (listened for and dismissed, because Playwright dismisses them silently when nothing listens — precisely the blind proceed the brief warns against). Failed assertions report page state rather than a bare verdict. The weak point is wait strategy: a `textPresent` wait that never satisfies fails as a locator problem though the locator is fine.

4. Heterogeneity & multi-tenant

`Surface` is the seam and `PlaywrightSurface` its only implementation. A frameset-era app would handle frame traversal inside `observe()` and `act()` with everything above unchanged; a desktop app would swap in UI Automation or `AXUIElement` and coordinate input. `adjacent` is a walk over a generic element tree rather than a DOM trick, which is what makes it plausible off the web.

Falling to a weaker rung is not the same as succeeding. It is tempting to claim the ladder absorbs tenant variation; the discovery run argues otherwise, since for a member whose name did not match the recorded one, the ladder fell to a positional rung and clicked link number eight. It happened to be right, nothing in the system knew that, and nothing would have reported it had it been wrong. So the honest design is base artifacts keyed by vendor product and version rather than by tenant, tenant overlays carrying the labels, routes and extra dialogs a tenant adds, a way to detect which product version a tenant runs, and a rule that resolving below the recorded rung flags or escalates rather than proceeding silently. The run log already records the rung, so the signal exists; nothing acts on it.

5. Escalation & handoff

Six codes escalate: `timeout`, `session_expired`, `assertion_failed`, `navigation_failed`, `approval_required`, `unexpected_dialog`. The test is whether a human at the keyboard could do anything about it, so `locator_not_found` is excluded — it means an artifact needs re-recording, not a person fixing a bug by hand, once, invisibly — as is `app_error`, which nobody repairs from inside the session. Discovery does not escalate at all, reporting `interventionRaised: false` when a budget runs out; the case for escalating is as

strong as for replay and the machinery exists, but it was not wired up. A step recorded `irreversible` stops the run *before* the locator resolves.

Sessions move `idle` → `running` → `paused` → `handed_off` with validated transitions, each landing in an append-only ledger tagged by actor. `OperatorTakeover` streams a CDP screencast and forwards input, acking each frame only after the consumer resolves. **Control is a token checked per input**, not a stream opened once by whoever was entitled at the time: the case that matters is after hand-back, when the operator's tab is still open and a stray click must be refused rather than raced against a resuming run. Operator inputs land in the ledger as `actor: operator` — the kind of input and a character count for typing, never the characters. On hand-back the last successful step's checkpoints are re-asserted, and operator actions never feed back into artifacts, since folding a post-timeout re-login into a capability would produce one that logs in twice. The mechanism is real and tested; there is no operator UI, and handoff is driven from `apps/cli/src/handoff.ts` — the cut the brief explicitly allows.

6. Safety

`Policy` declares allowed origins and path prefixes, per-action-class rules, redacted field names and patterns, `irreversibleControlLabels` and two budgets. Origins alone are too coarse for a real tenant, where admin and money-movement screens usually share one. Classification asks what an action can commit: treating every click as a read was the hole, since "Post Transaction" and "Back to Search" are the same DOM event, so a click is a mutation when the control's name matches the label list. A `fill` stays a mutation because a field can post back on blur. The default is `mutate: confirm` — prompt on a terminal, deny where there is none — and replay refuses an `irreversible` step without `--approve-irreversible`, never resolving the locator.

Secrets live in artifacts by reference only. Redaction masks declared names and patterns at the write boundary, and a test replays with a real password and fails if it appears in the log. Screenshots mask on the page before capture, since pixels are beyond any string filter. **Limits:** masking is only as good as the labelling;

`operatorUserId` is not redacted; outcome rules are a model's hypothesis with a measured failure rate (§3), contained only because a wrong rule degrades to `failed` rather than to a confidently wrong answer; the label list is a word list and nothing detects a miss; a 200-with-error-page reads as an ordinary failure; and page content, including whatever PII is on screen, reaches a third-party model during discovery, with Ollama as the local alternative.

7. Cuts

Left out: retrieval in front of intent matching (`task` already matches on task *shape* in one model call, but every capability enters the prompt); per-tenant overrides; a desktop surface; context trimming and resumption in discovery; multi-run stability scoring; a vision fallback; and an operator takeover screen. Automatic re-login on session expiry was considered and rejected, since it means handling credentials mid-run and resuming a flow whose completed steps may already have committed something.

Next, in order. The first three are the faults §3 measured, and none needs a schema change. First, offer the shaping step the condition that can actually fire: `when: step_failed` already exists and is right for not-found, but the model was shown only the derived checkpoints, and it should also be told what outcomes the flow *could* produce rather than only what the happy path showed. Second, keep instance values out of locator rungs — the recorder knows what was parameterised but not that text arriving downstream of a parameterised fill is equally instance data, so such a rung should be demoted below the positional one or the ladder flagged for review. Third, derive a non-empty assertion for extraction targets, or better, treat an empty extraction as a retryable read so a slow grid becomes `recovered` rather than `type_coercion_failed`. Then per-tenant locator patches with a weak-rung rule, retrieval-backed intent matching, and multi-run stability reporting.